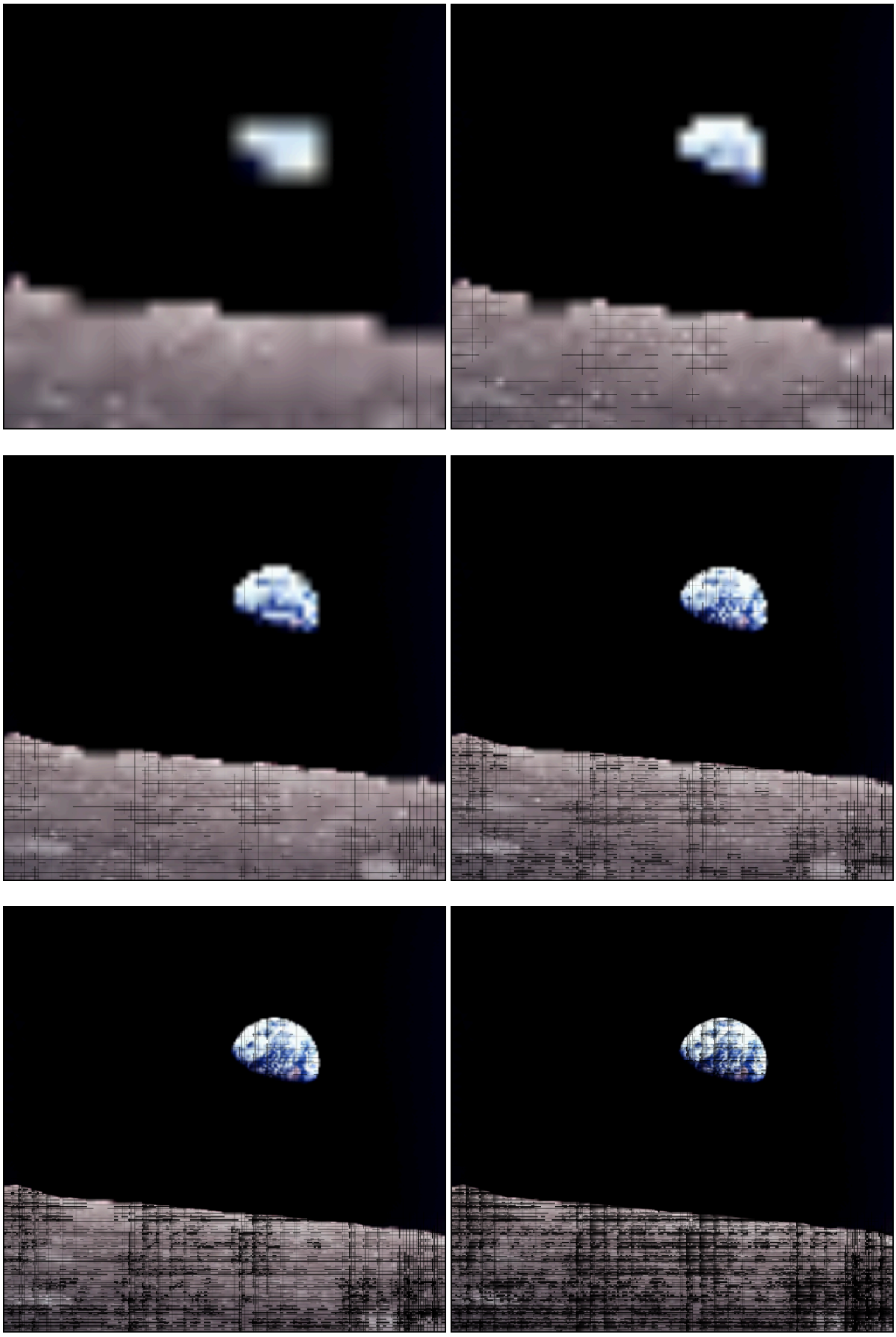
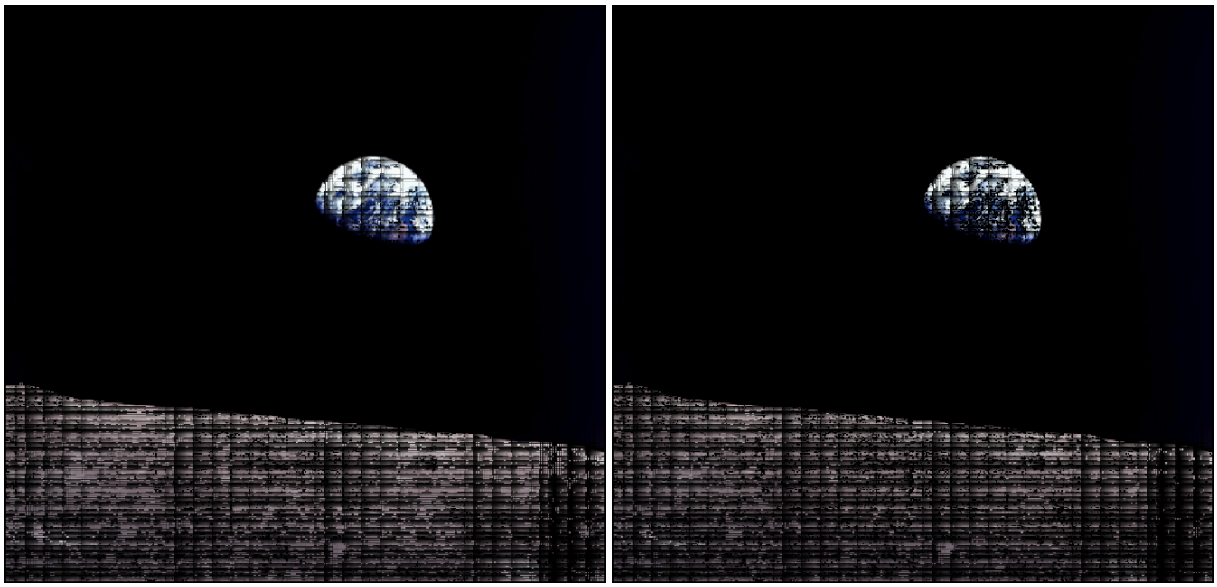
1. Użyta bitmapa



2. Sekwencja siatek po kolejnych iteracjach (od 1 do 10)







3. Ostateczna aproksymacja, bez siatek



4. Eksperyment tylko dla koloru czerwonego

4.1. bitmap_h_red

```
% computes r,g,b components of element in given point
function [r,g,b]=inpoint(xx,yy,width,hight,elem)
  f1 = fi1(xx,yy);
  f2 = fi2(xx,yy);
  f3 = fi3(xx,yy);
  f4 = fi4(xx,yy);

  r = vertexes(elem.dl).r * f1;
  r = r + vertexes(elem.ul).r * f2;
  r = r + vertexes(elem.dr).r * f3;
  r = r + vertexes(elem.ur).r * f4;
```

```
  r = floor(r);
  g = 0;
  b = 0;
% estimate relative error of interpolation over given element
function [error_r,error_g,error_b]=estimate_error(index)
  element = elements(index);
  dl = element.dl;
  ul = element.ul;
  dr = element.dr;
  ur = element.ur;

  xl = vertexes(dl).x;
  yd = vertexes(dl).y;
  xr = vertexes(ur).x;
  yu = vertexes(ur).y;

  elementWidth = xr - xl;
  elementHeigth = yu - yd;

  x1 = elementWidth/2.0 - elementWidth / (sqrt(3.0) * 2.0);
  x2 = elementWidth/2.0 + elementWidth / (sqrt(3.0) * 2.0);
  y1 = elementHeigth/2.0 - elementHeigth / (sqrt(3.0) * 2.0);
  y2 = elementHeigth/2.0 + elementHeigth / (sqrt(3.0) * 2.0);

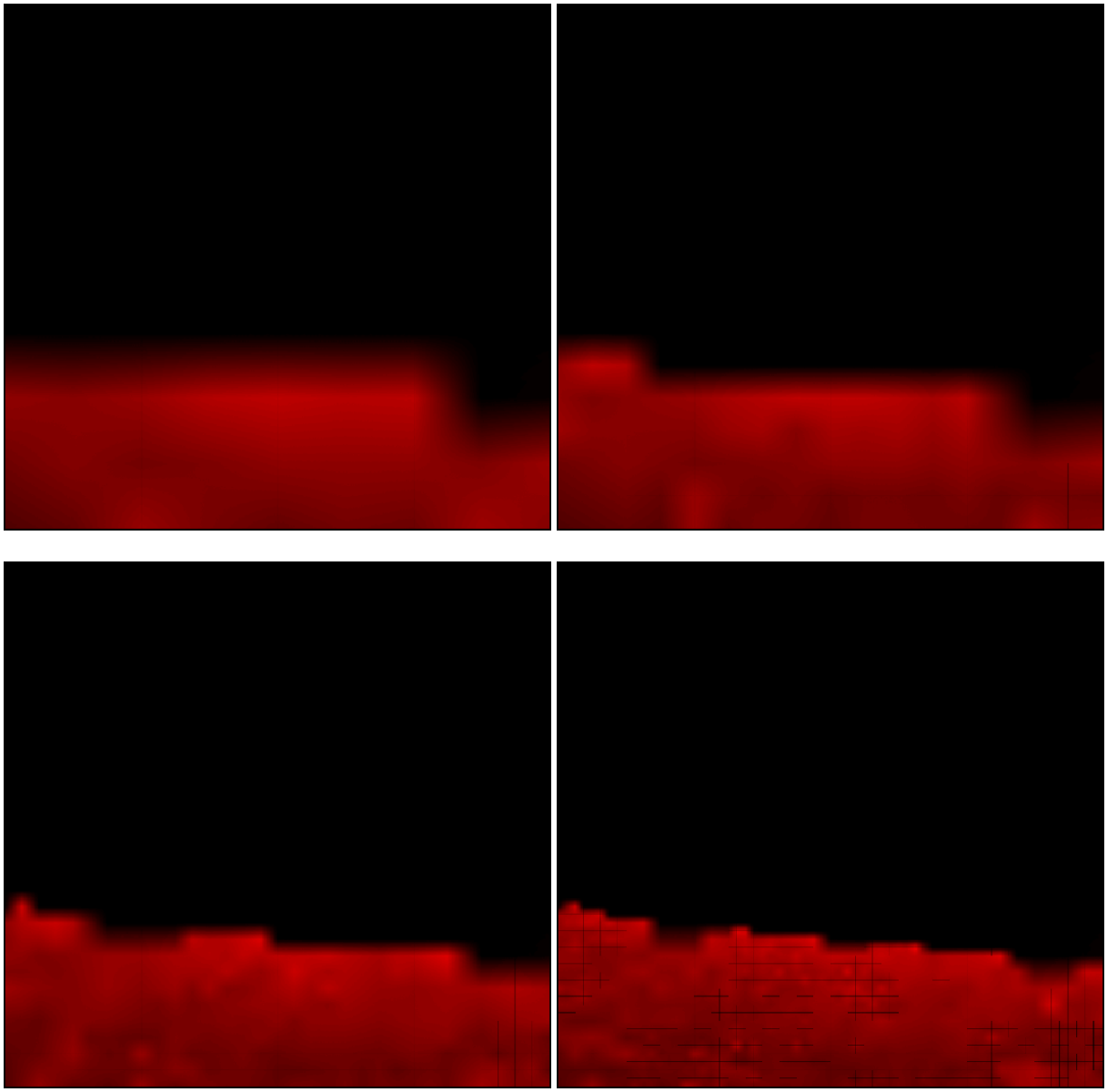
  x1 = floor(x1);
  x2 = floor(x2);
  y1 = floor(y1);
  y2 = floor(y2);

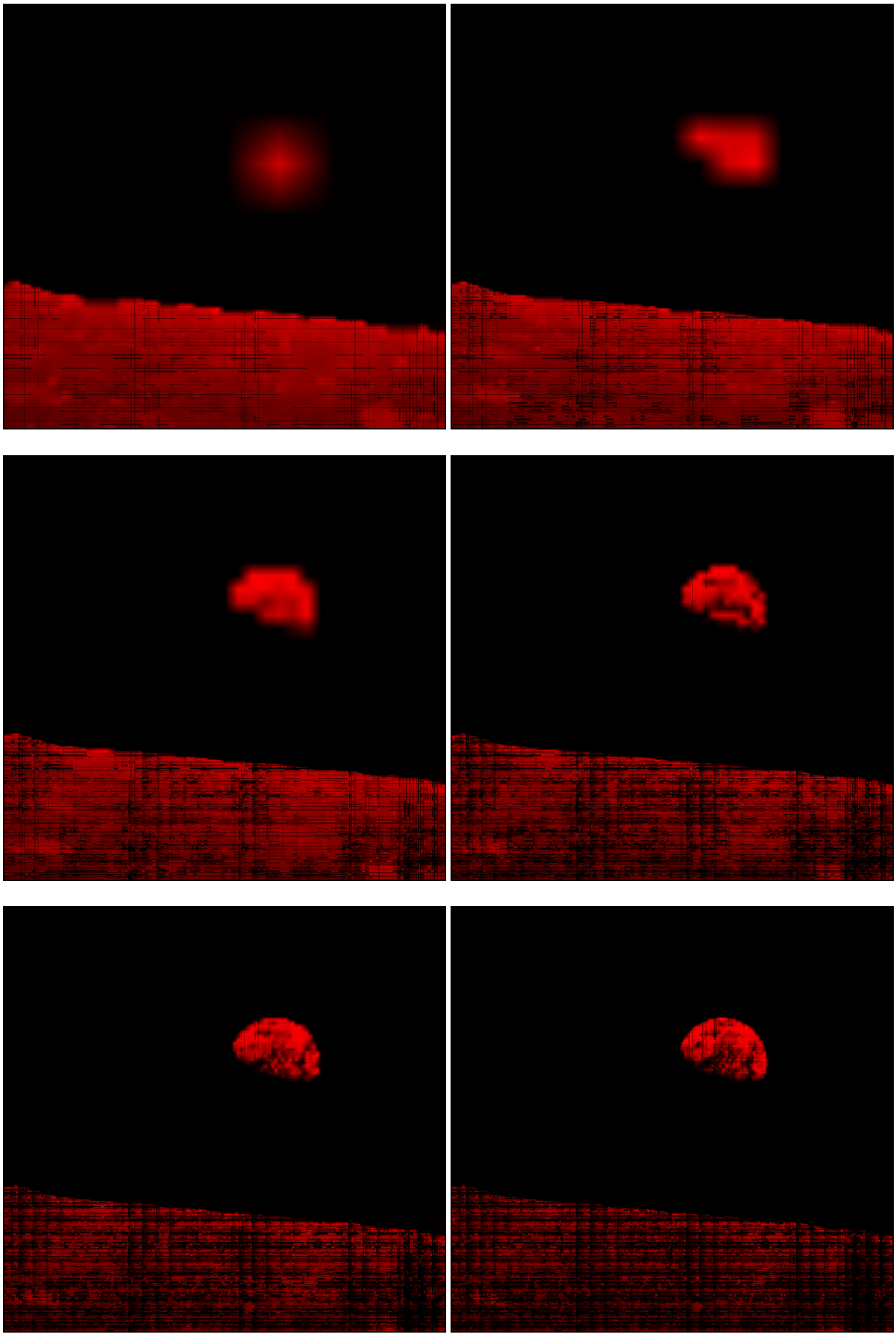
  [r1,g1,b1]=inpoint(x1,y1,elementWidth,elementHeigth,element);
  [r2,g2,b2]=inpoint(x1,y2,elementWidth,elementHeigth,element);
  [r3,g3,b3]=inpoint(x2,y1,elementWidth,elementHeigth,element);
  [r4,g4,b4]=inpoint(x2,y2,elementWidth,elementHeigth,element);

  r1 = r1 - RR(x1+xl,y1+yd);
  r2 = r2 - RR(x1+xl,y2+yd);
  r3 = r3 - RR(x2+xl,y1+yd);
  r4 = r4 - RR(x2+xl,y2+yd);

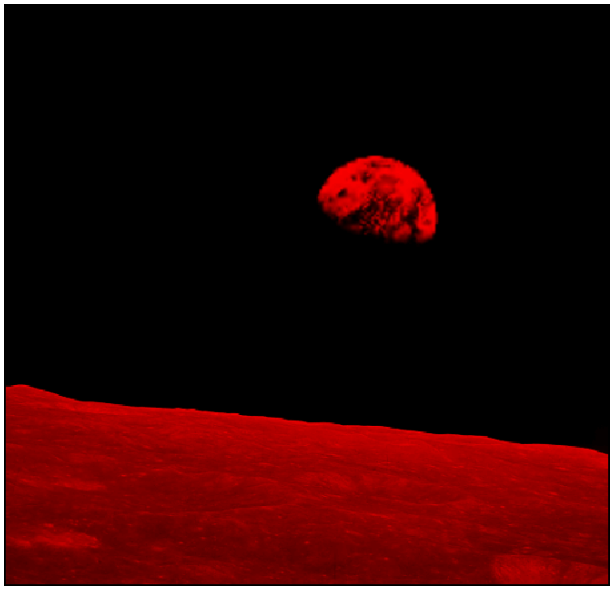
  error_r = r1*r1 + r2*r2 + r3*r3 + r4*r4;
  error_r = double(error_r);
  error_r = sqrt(error_r) * 100.0 / (255.0 * 2.0);
  error_g = 0;
  error_b = 0;
end
```

4.2. Wygenerowane siatki





4.3. Ostateczny obraz



5. Eksperyment tylko dla koloru zielonego

5.1. bitmap_h_green

```
% computes r,g,b components of element in given point
function [r,g,b]=inpoint(xx,yy,width,hight,elem)
  f1 = fi1(xx,yy);
  f2 = fi2(xx,yy);
  f3 = fi3(xx,yy);
  f4 = fi4(xx,yy);

  g = vertexes(elem.dl).g * f1;
  g = g + vertexes(elem.ul).g * f2;
  g = g + vertexes(elem.dr).g * f3;
  g = g + vertexes(elem.ur).g * f4;
  g = floor(g);
  r = 0;
  b = 0;
```

```
% estimate relative error of interpolation over given element
function [error_r,error_g,error_b]=estimate_error(index)
  element = elements(index);
  dl = element.dl;
  ul = element.ul;
  dr = element.dr;
  ur = element.ur;

  xl = vertexes(dl).x;
  yd = vertexes(dl).y;
  xr = vertexes(ur).x;
  yu = vertexes(ur).y;

  elementWidth = xr - xl;
  elementHeigth = yu - yd;
```

```
  x1 = elementWidth/2.0 - elementWidth / (sqrt(3.0) * 2.0);
  x2 = elementWidth/2.0 + elementWidth / (sqrt(3.0) * 2.0);
  y1 = elementHeigth/2.0 - elementHeigth / (sqrt(3.0) * 2.0);
  y2 = elementHeigth/2.0 + elementHeigth / (sqrt(3.0) * 2.0);

  x1 = floor(x1);
  x2 = floor(x2);
  y1 = floor(y1);
  y2 = floor(y2);

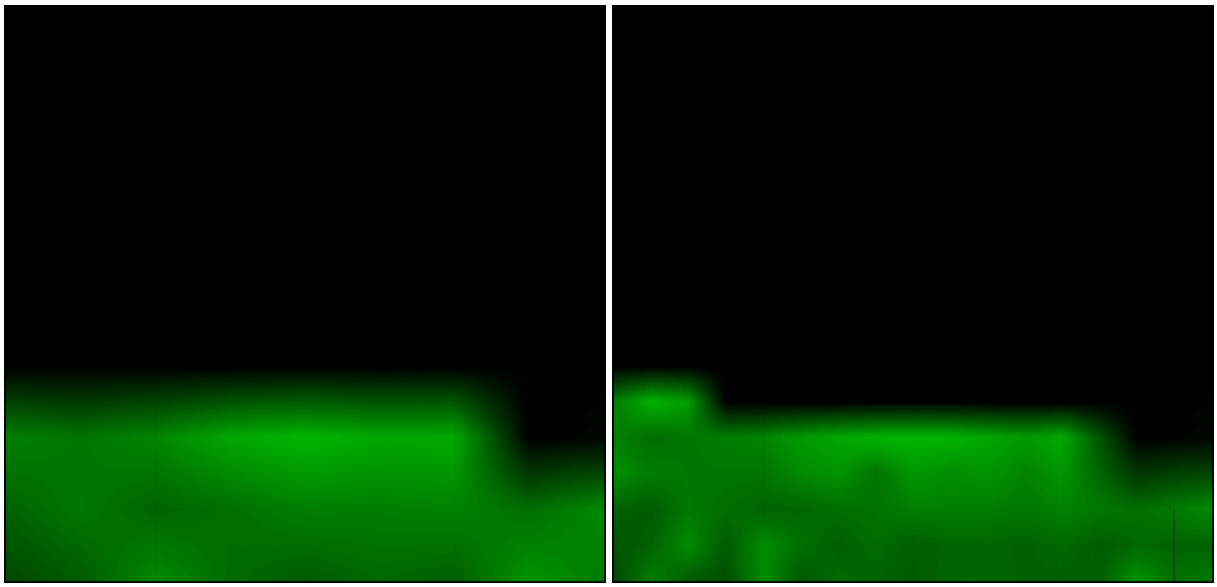
  [r1,g1,b1]=inpoint(x1,y1,elementWidth,elementHeigth,element);
  [r2,g2,b2]=inpoint(x1,y2,elementWidth,elementHeigth,element);
  [r3,g3,b3]=inpoint(x2,y1,elementWidth,elementHeigth,element);
  [r4,g4,b4]=inpoint(x2,y2,elementWidth,elementHeigth,element);

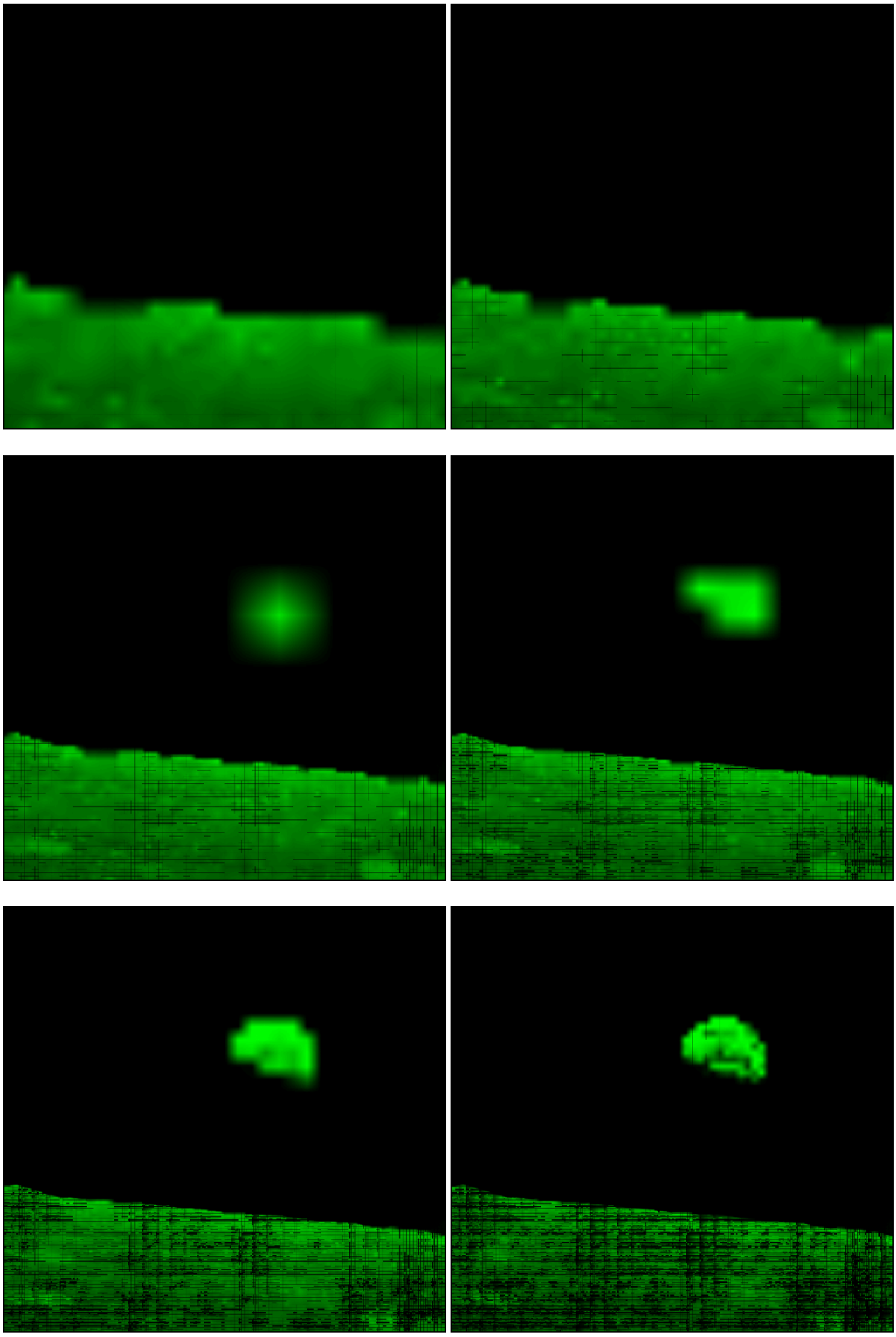
  g1 = g1 - GG(x1+xl,y1+yd);
  g2 = g2 - GG(x1+xl,y2+yd);
  g3 = g3 - GG(x2+xl,y1+yd);
  g4 = g4 - GG(x2+xl,y2+yd);

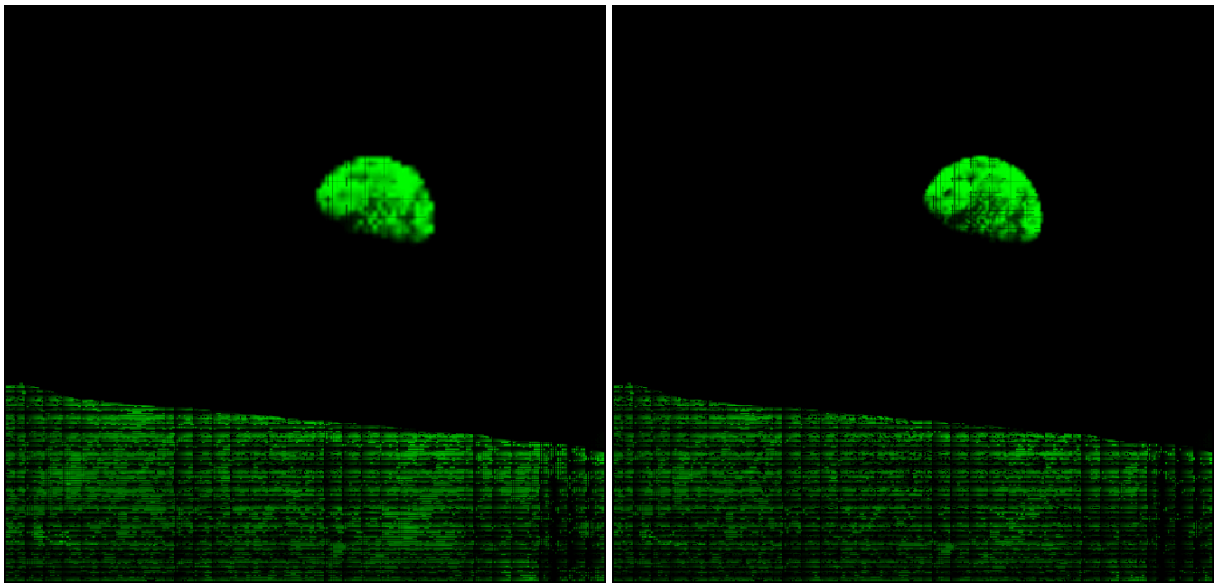
  error_g = g1*g1 + g2*g2 + g3*g3 + g4*g4;
  error_g = double(error_g);
  error_g = sqrt(error_g) * 100.0 / (255.0 * 2.0);

  error_r = 0;
  error_b = 0;
end
```

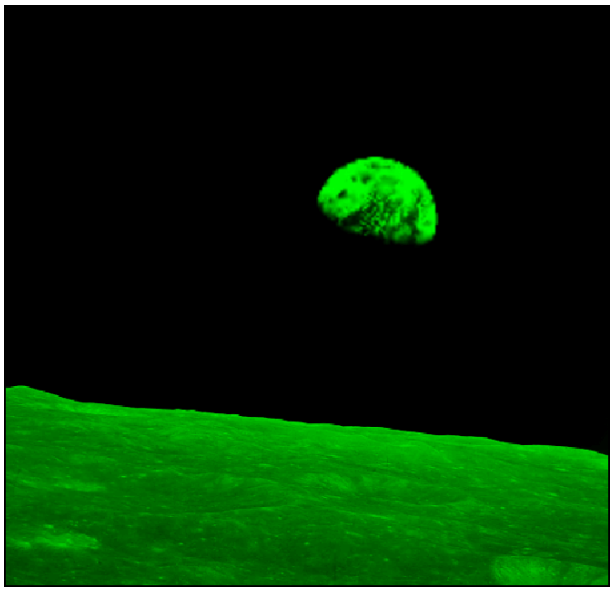
5.2. Wygenerowane siatki







5.3. Ostateczny obraz



6. Eksperyment tylko dla koloru niebieskiego

6.1. bitmap_h_blue

```
% computes r,g,b components of element in given point
function [r,g,b]=inpoint(xx,yy,width,hight,elem)
  f1 = fi1(xx,yy);
  f2 = fi2(xx,yy);
  f3 = fi3(xx,yy);
  f4 = fi4(xx,yy);

  b = vertexes(elem.dl).b * f1;
  b = b + vertexes(elem.ul).b * f2;
  b = b + vertexes(elem.dr).b * f3;
  b = b + vertexes(elem.ur).b * f4;
```

```
  b = floor(b);
  r = 0;
  g = 0;
% estimate relative error of interpolation over given element
function [error_r,error_g,error_b]=estimate_error(index)
  element = elements(index);
  dl = element.dl;
  ul = element.ul;
  dr = element.dr;
  ur = element.ur;

  xl = vertexes(dl).x;
  yd = vertexes(dl).y;
  xr = vertexes(ur).x;
  yu = vertexes(ur).y;

  elementWidth = xr - xl;
  elementHeigth = yu - yd;

% interpolate using L2 norm and Gaussian quadrature rule
  x1 = elementWidth/2.0 - elementWidth / (sqrt(3.0) * 2.0);
  x2 = elementWidth/2.0 + elementWidth / (sqrt(3.0) * 2.0);
  y1 = elementHeigth/2.0 - elementHeigth / (sqrt(3.0) * 2.0);
  y2 = elementHeigth/2.0 + elementHeigth / (sqrt(3.0) * 2.0);

  x1 = floor(x1);
  x2 = floor(x2);
  y1 = floor(y1);
  y2 = floor(y2);

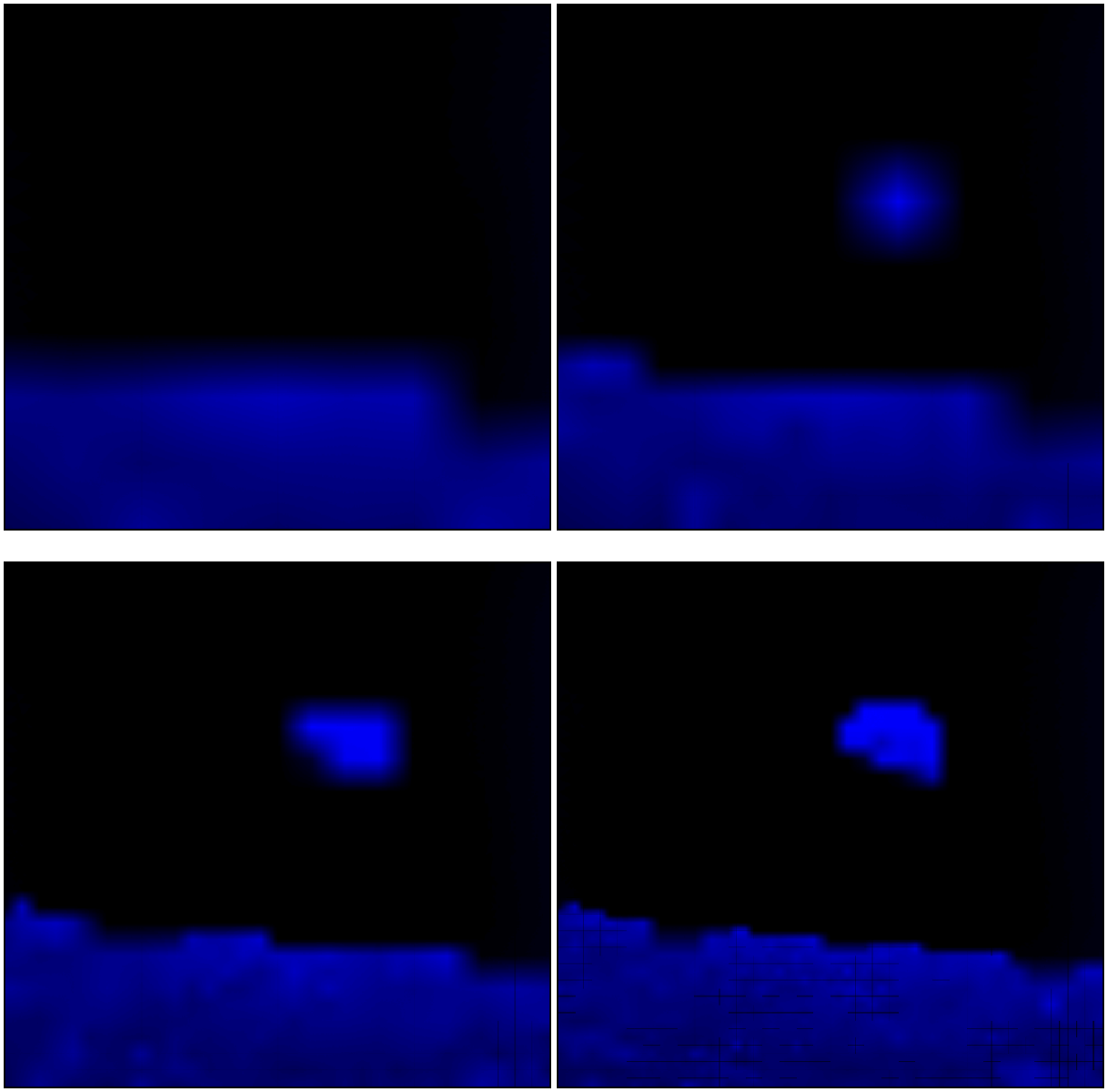
  [r1,g1,b1]=inpoint(x1,y1,elementWidth,elementHeigth,element);
  [r2,g2,b2]=inpoint(x1,y2,elementWidth,elementHeigth,element);
  [r3,g3,b3]=inpoint(x2,y1,elementWidth,elementHeigth,element);
  [r4,g4,b4]=inpoint(x2,y2,elementWidth,elementHeigth,element);

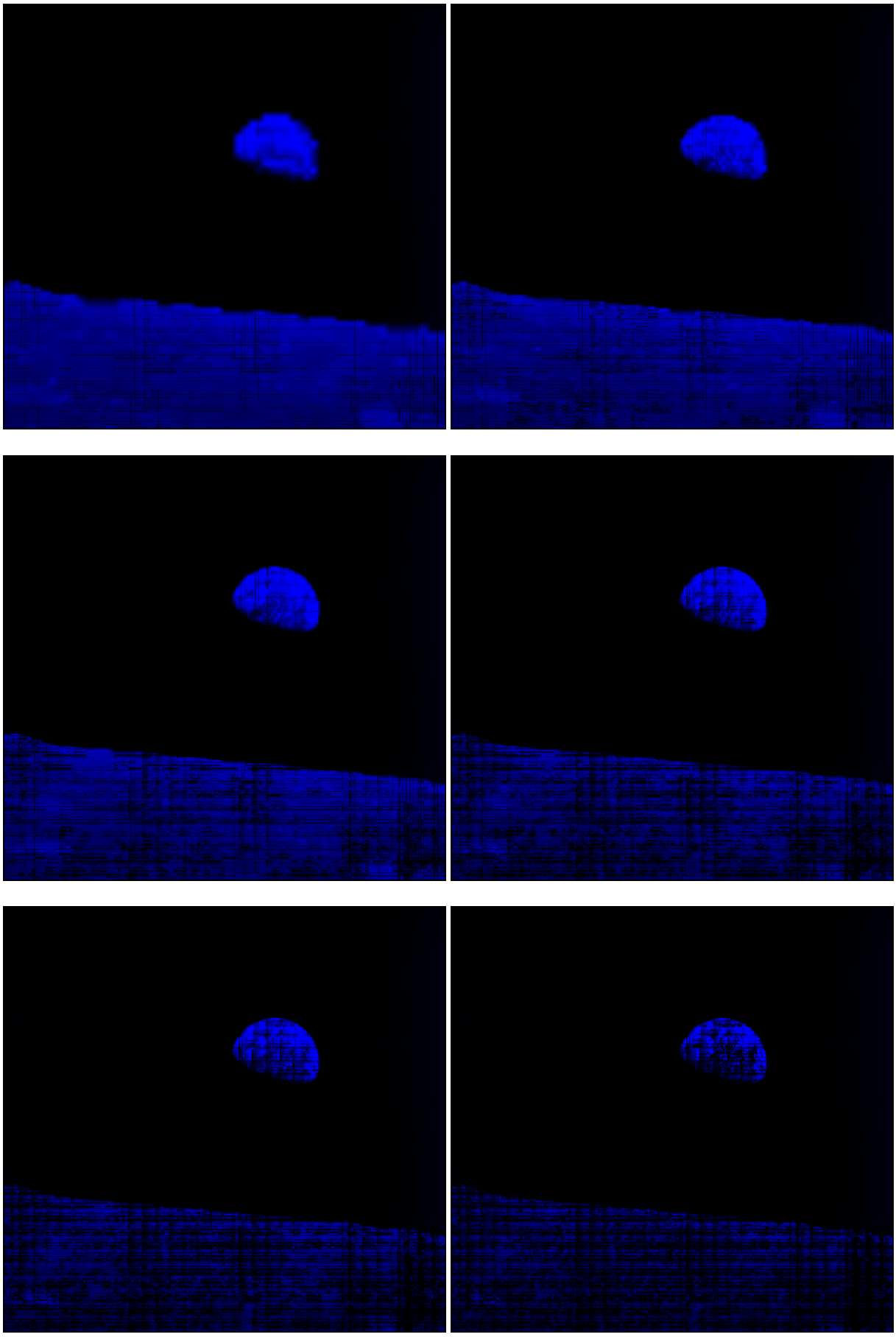
  b1 = b1 - BB(x1+xl,y1+yd);
  b2 = b2 - BB(x1+xl,y2+yd);
  b3 = b3 - BB(x2+xl,y1+yd);
  b4 = b4 - BB(x2+xl,y2+yd);

  error_b = b1*b1 + b2*b2 + b3*b3 + b4*b4;
  error_b = double(error_b);
  error_b = sqrt(error_b) * 100.0 / (255.0 * 2.0);

  error_r = 0;
  error_g = 0;
end
```

6.2. Wygenerowane siatki





6.3. Ostateczny obraz

